Proyecto Integrador – Primer Parcial

Tópicos y Programación Avanzada

Cada grupo construirá una aplicación completa que resuelve un problema real con texto: desde los datos crudos hasta una interfaz web usable, pasando por el proceso de NLP, uno o dos modelos de machine learning, un pipeline reproducible y un backend. La nota es grupal e individual: cada integrante defiende lo que hizo según su historial de commits y realiza una modificación en vivo el día de la presentación.

1. Datos generales

Campo	Detalle
Materia	Tópicos y Programación Avanzada
Tipo de trabajo	Proyecto integrador grupal
Integrantes por grupo	3 a 4 estudiantes
Fecha de presentación	3 de noviembre
Fecha límite de entrega	Día anterior a la presentación, 23:59 (último commit en el repositorio)
Modalidad de evaluación	Exposición grupal + defensa individual + modificación en vivo

2. Contexto y propósito

En la materia se trabajó el proceso de NLP (normalización, tokenización, stop words, stemming, lematización, n-gramas y TF-IDF) y ocho modelos de machine learning (regresión lineal, regresión logística, Naive Bayes, KNN, K-means, árbol de decisión, Random Forest y XGBoost). Este proyecto integra el proceso completo de NLP con uno o dos de esos modelos, elegidos por el grupo, en un solo producto de software funcional.

El propósito no es solo que el sistema funcione, sino que cada integrante entienda y pueda justificar cada decisión técnica: por qué se eligió un preprocesamiento, por qué el modelo elegido es adecuado y cómo llega una predicción desde el frontend hasta el modelo y de vuelta.

3. Objetivos

Objetivo general. Desarrollar una aplicación web completa que resuelva un problema basado en texto, integrando un pipeline de NLP y uno o dos modelos de machine learning, expuestos mediante un backend y consumidos desde un frontend.

Objetivos específicos.

1. Construir y justificar un pipeline de preprocesamiento de texto que aplique todas las técnicas de NLP vistas en clase.
2. Seleccionar uno o dos modelos de machine learning adecuados al problema, entrenarlos, evaluarlos con métricas y justificar la elección.
3. Empaquetar el preprocesamiento y los modelos en un pipeline reproducible que se use igual en entrenamiento y en producción.

4. Exponer los modelos mediante una API y construir un frontend que permita usarlos e interpretar sus resultados.
5. Trabajar colaborativamente con Git, dejando trazabilidad clara de la contribución de cada integrante.

4. Problema a resolver y temas sugeridos

Cada grupo elige un dominio con datos de texto y define una tarea principal: clasificar, estimar un valor numérico o agrupar textos. Luego escoge **uno o dos modelos** de los vistos en clase que sean adecuados para esa tarea y justifica la elección.

Tema sugerido	Tarea	Modelos adecuados (orientativo)
Reseñas de productos, hoteles o restaurantes	Clasificar el sentimiento	Regresión logística, Naive Bayes, Random Forest
Reseñas de productos, hoteles o restaurantes	Estimar la puntuación (1 a 5)	Regresión lineal, Random Forest, XGBoost
Tickets de soporte o mesa de ayuda	Clasificar área o prioridad	Naive Bayes, regresión logística, XGBoost
Ofertas laborales	Estimar el salario	Regresión lineal, Random Forest, XGBoost
Reclamos ciudadanos a una institución	Clasificar la dependencia responsable	Árbol de decisión, Random Forest, KNN
Noticias o publicaciones en redes	Agrupar por tema sin etiquetas	K-means

La columna de modelos es una guía: el grupo puede elegir otro modelo visto en clase si justifica que es adecuado para su tarea.

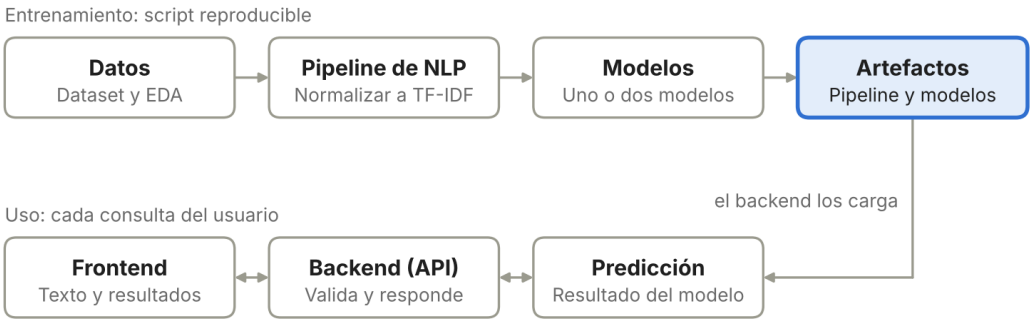
Reglas para el dataset:

- Puede ser público o recolectado por el grupo; se documenta su origen, licencia y fecha de obtención.
- Se recomienda que esté en español, ya que el pipeline de NLP (stop words, stemming, lematización) debe adaptarse al idioma.
- Tamaño mínimo sugerido: [completar, por ejemplo 1.000 registros etiquetados].
- Un tema propio fuera de la tabla es válido si el docente lo aprueba antes de [completar fecha].

5. Requisitos técnicos

El sistema tiene cinco capas obligatorias: datos, pipeline de NLP, modelos, backend y frontend. El preprocesamiento usado al entrenar debe ser exactamente el mismo que se usa al predecir desde la API.

El mismo pipeline entrena los modelos y atiende las predicciones



El script de entrenamiento guarda el pipeline y los modelos; el backend los carga y nunca reimplementa el preprocesamiento.

5.1 Datos y análisis exploratorio

- Análisis exploratorio: distribución de clases, longitud de los textos, palabras más frecuentes y valores faltantes.
- División entrenamiento / prueba estratificada y con semilla fija; el conjunto de prueba no se usa para ajustar nada.
- Tratamiento justificado del desbalance de clases, si existe.

5.2 Pipeline de NLP

Cada paso se implementa, se muestra con un ejemplo antes / después y se justifica.

Paso	Qué se exige
Normalización	Minúsculas, tildes, puntuación, números, URLs y emojis; justificar qué se elimina y qué se conserva
Tokenización	Separar el texto en tokens; mostrar casos difíciles del dominio
Stop words	Lista en español; justificar palabras agregadas o quitadas (por ejemplo, la negación “no” en sentimiento)
Stemming y lematización	Implementar ambos y comparar su efecto en las métricas para elegir uno
N-gramas	Comparar al menos unigramas contra unigramas + bigramas
TF-IDF	Vectorización con parámetros documentados (tamaño de vocabulario, frecuencias mínima y máxima)

5.3 Modelos de machine learning

Cada grupo usa uno o dos modelos. Con dos modelos, se comparan y se justifica cuál queda en producción. Con un solo modelo, se compara contra una línea base simple (por ejemplo, predecir siempre la clase más frecuente o el promedio) para demostrar que aporta valor.

Tipo de modelo
Clasificación (regresión logística, Naive Bayes, KNN, árbol de decisión, Random Forest, XGBoost)
Regresión (regresión lineal; Random Forest o XGBoost en modo regresión)

Tipo de modelo
Agrupamiento (K-means)

- Justificación de por qué el modelo elegido es adecuado para el problema y los datos.
- Ajuste de hiperparámetros con validación cruzada en al menos un modelo.
- Análisis de errores: ejemplos donde el modelo falla y una explicación de por qué.
- Todos los integrantes deben poder explicar cómo funciona internamente el modelo elegido.

5.4 Pipeline reproducible

- Un script de entrenamiento que, con un solo comando, va desde los datos crudos hasta los modelos guardados.
- El vectorizador y los modelos se serializan y el backend los carga; el backend no reimplementa el preprocesamiento.
- Dependencias fijadas con sus versiones y semilla aleatoria fija para que los resultados se puedan repetir.

5.5 Backend

- API REST con framework libre (Flask, FastAPI, Django u otro, justificado).
- Endpoints mínimos: predicción del modelo en producción (clase con su probabilidad, valor estimado o grupo asignado, según la tarea), métricas del modelo y un endpoint de estado del servicio.
- Validación de entradas (texto vacío, demasiado largo, formato incorrecto) y mensajes de error claros.
- Documentación de los endpoints y al menos pruebas automatizadas básicas.

5.6 Frontend

- Interfaz web con framework libre que consuma la API.
- El usuario ingresa un texto (o sube un archivo con varios) y ve el resultado del modelo: la clase con su probabilidad, el valor estimado o el grupo asignado.
- Vista del preprocesamiento paso a paso: texto normalizado, tokens, stop words eliminadas y lemas o raíces.
- Panel con las métricas del modelo y, si se usaron dos, su comparación.

5.7 Ejecución

- README con instrucciones para instalar y levantar el sistema completo desde cero.
- Contenedores (por ejemplo, Docker) o despliegue en la nube son opcionales y suman como valor agregado.

6. Organización del equipo y uso del repositorio

El repositorio es la evidencia principal de la contribución individual: lo que no está en el historial de commits de una persona no cuenta como trabajo suyo.

6.1 Roles sugeridos

Rol	Responsabilidad principal
Datos y NLP	Dataset, análisis exploratorio y pipeline de preprocesamiento
Modelado	Entrenamiento, ajuste y comparación de los modelos
Backend	API, carga de modelos, validaciones y pruebas

Rol	Responsabilidad principal
Frontend	Interfaz, consumo de la API y visualizaciones

En grupos de 4 o menos. Los roles orientan el trabajo pero no lo encierran: **cada integrante debe tener al menos un commit significativo en la parte de NLP o machine learning**, porque todos serán preguntados sobre ello.

6.2 Reglas del repositorio

1. Un repositorio por grupo en GitHub o GitLab, con el docente agregado como colaborador desde el inicio del proyecto.
2. Cada integrante trabaja con su propia cuenta y con su nombre y correo configurados en Git. No se permiten cuentas compartidas ni commits hechos en nombre de otro.
3. El trabajo en pareja se registra con la línea `Co-authored-by:` en el mensaje del commit.
4. Una rama por funcionalidad, integrada a la rama principal mediante pull request revisado por otro integrante.
5. Mensajes de commit descriptivos (qué se hizo y por qué); se recomienda el formato de Conventional Commits.
6. Commits frecuentes y repartidos en el tiempo. Un único commit gigante el último día se considera una señal de alerta y se revisará con más detalle.
7. No reescribir el historial de la rama principal (force push, squash de todo el proyecto), porque se pierde la trazabilidad de autoría.
8. Archivo `CONTRIBUCIONES.md` con lo que hizo cada integrante; debe coincidir con el historial de commits.

6.3 Estructura sugerida

```
proyecto/
├── data/           # datos crudos y procesados (o script de descarga)
├── notebooks/      # análisis exploratorio y experimentos
├── src/
│   ├── nlp/        # pipeline de preprocesamiento
│   └── models/     # entrenamiento y evaluación
├── api/            # backend
├── frontend/       # interfaz web
├── tests/
├── docs/           # informe técnico
├── README.md
└── CONTRIBUCIONES.md
```

7. Entregables y fecha de entrega

Todo se entrega en el repositorio hasta las 23:59 del día anterior a la presentación. El último commit de la rama principal antes de esa hora es la versión evaluada; los commits posteriores no se consideran, salvo los que se hagan en la defensa a pedido del docente.

Al entregar, el grupo crea la etiqueta (tag) `entrega-final` sobre ese commit.

- ☐ Código fuente completo: pipeline de NLP, entrenamiento, backend y frontend
- ☐ Modelos serializados, o el script que los regenera con un solo comando
- ☐ Notebook de análisis exploratorio y experimentos

- ☐ Informe técnico breve en `docs/`: problema, dataset, decisiones de NLP, tabla comparativa de modelos y arquitectura
- ☐ README con instrucciones de instalación y ejecución
- ☐ `CONTRIBUCIONES.md` con la contribución de cada integrante
- ☐ Diapositivas de la presentación
- ☐ Etiqueta `entrega-final` creada

8. Presentación y defensa

El día de la presentación el grupo clona su repositorio desde la etiqueta `entrega-final` y levanta el sistema funcionando. Los tiempos indicados son una propuesta: [ajustar según la duración de la clase].

1. **Exposición grupal (aprox. 10 minutos):** problema, dataset, arquitectura y demostración del sistema completo.
2. **Explicación individual (aprox. 3 minutos por integrante):** cada integrante explica su parte apoyándose en sus propios commits.
3. **Preguntas individuales:** el docente revisa el historial y pregunta a cada integrante sobre commits concretos que hizo (qué cambió, por qué, qué alternativas consideró). También puede preguntar sobre cualquier parte del proyecto.
4. **Modificación en vivo:** el docente asigna al grupo un cambio pequeño relacionado con su área. Lo implementa en ese momento, lo ejecuta y lo sube con un commit propio.

Ejemplos de modificaciones que se pueden solicitar:

Área	Ejemplo de modificación
NLP	Agregar o quitar una stop word y mostrar cómo cambian los resultados
NLP	Cambiar el rango de n-gramas o cambiar stemming por lematización y reentrenar
Modelado	Cambiar un hiperparámetro del modelo elegido, reentrenar y comparar métricas
Modelado	Calcular e interpretar una métrica adicional del modelo (por ejemplo, la matriz de confusión o el error por categoría)
Backend	Agregar un endpoint o una validación nueva (por ejemplo, rechazar textos de menos de 3 palabras)
Frontend	Mostrar un dato nuevo de la API o cambiar una visualización

9. Evaluación

La nota final de cada estudiante combina una parte grupal (40 puntos, igual para todo el grupo) y una parte individual (60 puntos, propia de cada integrante). Los pesos son una propuesta ajustable.

Componente	Criterio	Puntos	Qué se observa
Grupal	Pipeline de NLP	10	Todos los pasos implementados, comparados y justificados
Grupal	Modelos y comparación	10	Elección justificada de uno o dos modelos, métricas correctas, comparación y análisis de errores
Grupal	Integración y funcionamiento	10	Pipeline reproducible, API y frontend funcionando de punta a punta
Grupal	Repositorio y documentación	5	README, informe, estructura, uso de ramas y pull requests
Grupal	Exposición	5	Claridad, demostración y manejo del tiempo

Componente	Criterio	Puntos	Qué se observa
Individual	Contribución en el repositorio	20	Relevancia, calidad y constancia de sus commits a lo largo del proyecto
Individual	Explicación de su parte	15	Explica con propiedad lo que hizo y por qué
Individual	Respuestas a preguntas	10	Responde sobre sus commits y sobre el proyecto en general
Individual	Modificación en vivo	15	Completa el cambio, lo ejecuta, interpreta el resultado y lo sube con un commit
	Total	**100**	

Criterios para la contribución en el repositorio:

- Se valora la relevancia y la calidad de los commits, no su cantidad. Cambios de formato, comentarios o archivos generados no cuentan como aporte sustancial.
- Si un integrante no puede explicar un commit propio, ese commit no se considera en su nota.
- Una contribución desproporcionadamente baja respecto al resto del grupo se refleja solo en la nota individual de esa persona.

10. Normas y honestidad académica

- Se puede reutilizar código de librerías y documentación citando la fuente en el código o en el informe.
- Copiar el proyecto de otro grupo o de un repositorio externo sin atribución se considera plagio y se sanciona según el reglamento de la universidad.
- Falsificar la autoría de commits (usar la cuenta de otro, reescribir autores) anula la nota individual de los involucrados.
- Uso de asistentes de IA: permitido con declaración en el informe. En cualquier caso, cada integrante debe poder explicar y modificar todo el código que subió.